

PRACTICE EXERCISES OF THE MICROPROCESSORS & MICROCONTROLLERS

Instructor: The Tung Than

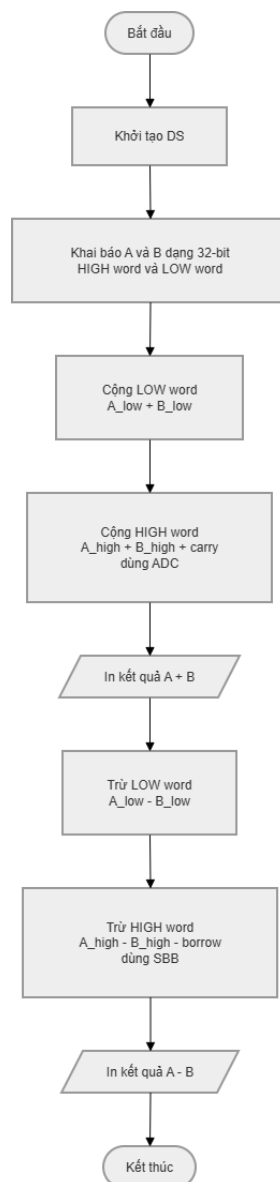
Student's name: Nguyễn Đông Quân

Student code: 24521438

PRACTICE REPORT NO #6: IO PROCESSING, CALCULATION AND MEMORY ON THE 8086 MICROPROCESSOR

I. Content 1:

Write a program that adds/subtracts 2 32-bit numbers



Flowchart bài 1

```
org 100h

jmp start

; A = 12345678h
a_low dw 5678h
a_high dw 1234h

; B = 00021111h
b_low dw 1111h
b_high dw 0002h

msg1 db 13,10,'A + B = $'
msg2 db 13,10,'A - B = $'

start:
    mov ax, cs
    mov ds, ax

    ; A + B
    lea dx, msg1
    call puts

    mov ax, a_low
    add ax, b_low
    mov cx, ax

    mov ax, a_high
    adc ax, b_high

    call print_hex

    mov dl, '.'
    call putc

    mov ax, cx
    call print_hex
```

```
; A - B
lea dx, msg2
call puts

mov ax, a_low
sub ax, b_low
mov cx, ax

mov ax, a_high
sbb ax, b_high

call print_hex

mov dl, ':'
call putc

mov ax, cx
call print_hex

mov ax, 4C00h
int 21h

puts proc
    mov ah, 09h
    int 21h
    ret
puts endp

putc proc
    mov ah, 02h
    int 21h
    ret
putc endp

print_hex proc
    push ax
    push bx
    push cx
```

```
push dx

mov bx, ax
mov ch, 4

hex_loop:
    mov cl, 4
    rol bx, cl

    mov dl, bl
    and dl, 0Fh

    cmp dl, 9
    jbe digit

    add dl, 7

digit:
    add dl, '0'
    call putc

    dec ch
    jnz hex_loop

pop dx
pop cx
pop bx
pop ax
ret
print_hex endp
```

Giải thích chi tiết:

1. Khai báo dữ liệu và khởi tạo DS

```
org 100h

jmp start
```

```

; A = 12345678h
a_low dw 5678h
a_high dw 1234h

; B = 00021111h
b_low dw 1111h
b_high dw 0002h

msg1 db 13,10,'A + B = $'
msg2 db 13,10,'A - B = $'

start:
    mov ax, cs
    mov ds, ax

```

- Chương trình dùng org 100h để chạy dưới dạng chương trình .COM trong emu8086. Vì 8086 xử lý dữ liệu chủ yếu theo thanh ghi 16-bit, nên mỗi số 32-bit được tách thành hai phần: phần thấp LOW word và phần cao HIGH word.

- Số A = 12345678h được tách thành a_high = 1234h và a_low = 5678h. Số B = 00021111h được tách thành b_high = 0002h và b_low = 1111h. Việc tách như vậy giúp chương trình có thể cộng/trừ số 32-bit bằng các lệnh 16-bit của 8086.

- Hai chuỗi msg1 và msg2 dùng để in thông báo kết quả ra màn hình. Lệnh mov ax, cs và mov ds, ax dùng để khởi tạo thanh ghi đoạn dữ liệu DS, giúp chương trình truy cập đúng các biến đã khai báo trong bộ nhớ.

2. Cộng và trừ hai số 32-bit

```

; A + B
lea dx, msg1
call puts

mov ax, a_low
add ax, b_low
mov cx, ax

mov ax, a_high
adc ax, b_high

call print_hex

```

```

mov dl, ':'
call putc

mov ax, cx
call print_hex

```

- Đầu tiên, chương trình in chuỗi $A + B =$ bằng cách đưa địa chỉ msg1 vào DX, sau đó gọi thủ tục puts.

- Để cộng hai số 32-bit, chương trình cộng phần thấp trước: $a_low + b_low$. Kết quả phần thấp được lưu tạm vào thanh ghi CX. Sau đó, chương trình cộng phần cao bằng lệnh ADC: $a_high + b_high + carry$. Lệnh ADC rất quan trọng vì nếu phép cộng phần thấp bị tràn thì cờ nhớ carry sẽ được cộng thêm vào phần cao.

- Sau khi tính xong, phần cao của kết quả đang nằm trong AX, nên chương trình gọi print_hex để in phần cao trước. Sau đó in dấu : để phân tách HIGH word và LOW word, rồi đưa phần thấp từ CX về AX để in tiếp. Vì vậy kết quả được in theo dạng HIGH:LOW, ví dụ 1236:6789.

```

; A - B
lea dx, msg2
call puts

mov ax, a_low
sub ax, b_low
mov cx, ax

mov ax, a_high
sbb ax, b_high

call print_hex

mov dl, ':'
call putc

mov ax, cx
call print_hex

mov ax, 4C00h
int 21h

```

- Phần trừ cũng được xử lý tương tự. Chương trình in chuỗi $A - B =$, sau đó trừ phần thấp trước bằng lệnh SUB: $a_low - b_low$. Kết quả phần thấp được lưu tạm vào CX.

- Tiếp theo, chương trình trừ phần cao bằng lệnh SBB: a_high - b_high - borrow. Lệnh SBB sẽ trừ thêm cờ mượn nếu phép trừ phần thấp cần mượn. Nhờ đó, phép trừ 32-bit được thực hiện đúng trên vi xử lý 8086.

- Cuối cùng, chương trình in phần cao, dấu :, rồi in phần thấp giống như phép cộng. Lệnh mov ax, 4C00h và int 21h dùng để kết thúc chương trình.

3. Các thủ tục in chuỗi, in ký tự và in số HEX

```
puts proc
    mov ah, 09h
    int 21h
    ret
puts endp
```

```
putc proc
    mov ah, 02h
    int 21h
    ret
putc endp
```

- Thủ tục puts dùng để in một chuỗi ký tự ra màn hình. Chuỗi cần in phải kết thúc bằng ký tự \$, đúng với chức năng AH = 09h của ngắt INT 21h.

- Thủ tục putc dùng để in một ký tự ra màn hình. Ký tự cần in được đặt trong thanh ghi DL, sau đó gọi ngắt INT 21h với chức năng AH = 02h.

```
print_hex proc
    push ax
    push bx
    push cx
    push dx

    mov bx, ax
    mov ch, 4

hex_loop:
    mov cl, 4
    rol bx, cl

    mov dl, bl
    and dl, 0Fh
```

```
    cmp dl, 9
    jbe digit

    add dl, 7

digit:
    add dl, '0'
    call putc

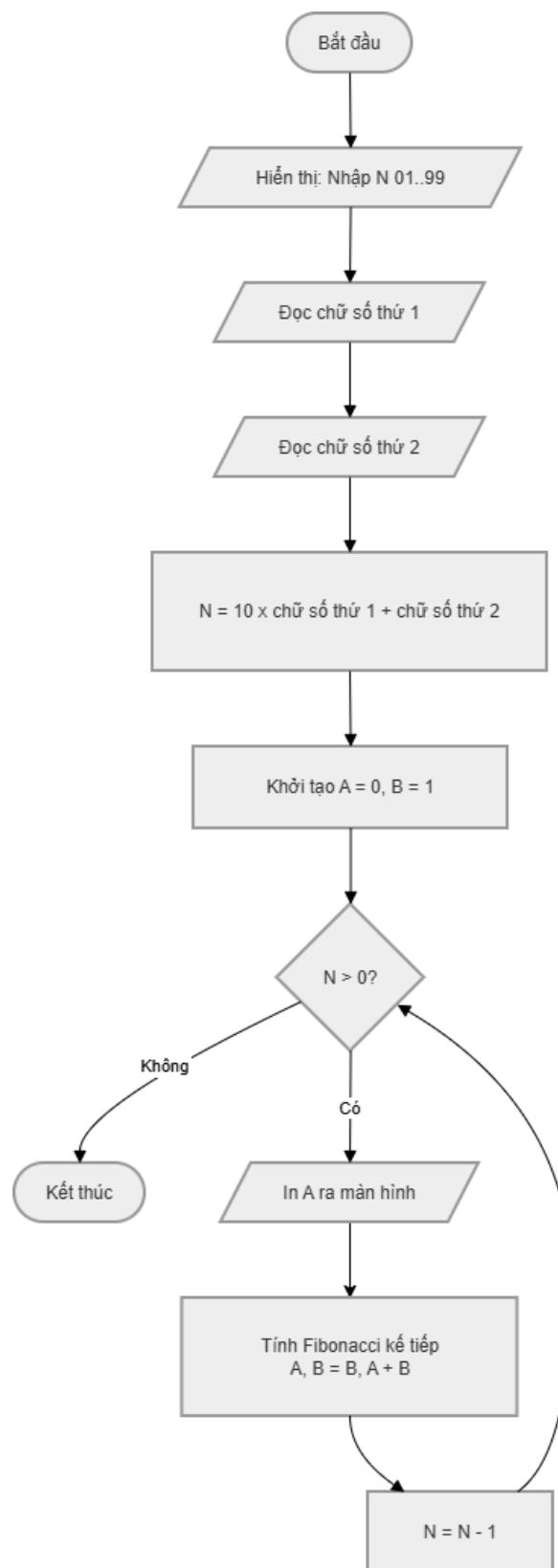
    dec ch
    jnz hex_loop

    pop dx
    pop cx
    pop bx
    pop ax
    ret
print_hex endp
```

- Thủ tục print_hex dùng để in giá trị 16-bit trong thanh ghi AX ra màn hình dưới dạng 4 chữ số hệ thập lục phân. Đầu tiên, giá trị trong AX được chuyển sang BX để xử lý. Thanh ghi CH = 4 được dùng làm bộ đếm vì một số 16-bit gồm 4 chữ số HEX.
- Trong mỗi vòng lặp, chương trình dùng ROL để xoay BX sang trái 4 bit, đưa từng nhóm 4 bit ra vị trí thấp. Sau đó dùng AND 0Fh để lấy đúng 4 bit thấp này. Nếu giá trị nằm trong khoảng 0 đến 9, chương trình cộng với '0' để đổi thành ký tự số. Nếu lớn hơn 9, chương trình cộng thêm 7 trước khi cộng '0' để tạo ra ký tự A đến F.
- Các lệnh push và pop được dùng để lưu và khôi phục các thanh ghi trước và sau khi in HEX, tránh làm ảnh hưởng đến chương trình chính.

II. Exercise 1:

Enter a 2-digit number N from the keyboard through the console screen. Print the first N Fibonacci numbers to the screen.



Flowchart bài 2

```
org 100h

jmp start

DIG equ 21

msg db 'Nhap N 01-99: $'

n db ?
a db DIG dup(0)
b db 1, (DIG-1) dup(0)

start:
    mov ax, cs
    mov ds, ax

    lea dx, msg
    mov ah, 09h
    int 21h

    mov ah, 01h
    int 21h
    sub al, '0'
    mov bl, al

    mov ah, 01h
    int 21h
    sub al, '0'

    mov ah, bl
    aad

    mov n, al
    call crlf

main_loop:
    call print_a
    call next_fib
```

```
    dec n
    jnz main_loop

    mov ax, 4C00h
    int 21h

print_a proc
    mov bx, DIG-1

find_digit:
    cmp byte ptr [a+bx], 0
    jne print_loop

    dec bx
    jns find_digit

    mov dl, '0'
    mov ah, 02h
    int 21h
    call crlf
    ret

print_loop:
    mov ah, 02h

print_digit:
    mov dl, [a+bx]
    add dl, '0'
    int 21h

    dec bx
    jns print_digit

    call crlf
    ret
print_a endp
```

```
next_fib proc
```

```
    xor si, si
```

```
    xor bl, bl
```

```
    mov cx, DIG
```

```
next_loop:
```

```
    mov al, [b+si]
```

```
    mov dl, al
```

```
    add al, [a+si]
```

```
    add al, bl
```

```
    xor bl, bl
```

```
    cmp al, 10
```

```
    jb no_carry
```

```
    sub al, 10
```

```
    inc bl
```

```
no_carry:
```

```
    mov [a+si], dl
```

```
    mov [b+si], al
```

```
    inc si
```

```
    loop next_loop
```

```
    ret
```

```
next_fib endp
```

```
crlf proc
```

```
    mov ah, 02h
```

```
    mov dl, 13
```

```
    int 21h
```

```
    mov dl, 10
```

```
    int 21h
```

```
ret  
crlf endp
```

Giải thích chi tiết:

1. Khai báo dữ liệu và nhập N

```
org 100h  
  
jmp start  
  
DIG equ 21  
  
msg db 'Nhap N 01-99: $'  
  
n db ?  
a db DIG dup(0)  
b db 1, (DIG-1) dup(0)  
  
start:  
    mov ax, cs  
    mov ds, ax  
  
    lea dx, msg  
    mov ah, 09h  
    int 21h  
  
    mov ah, 01h  
    int 21h  
    sub al, '0'  
    mov bl, al  
  
    mov ah, 01h  
    int 21h  
    sub al, '0'  
  
    mov ah, bl
```

```
aad
```

```
mov n, al
```

```
call crlf
```

- Chương trình khai báo DIG equ 21 để mỗi số Fibonacci có thể lưu tối đa 21 chữ số. Vì N lớn nhất là 99, chương trình cần in tối đa 99 số Fibonacci đầu tiên, tức từ F(0) đến F(98). Do F(98) có 21 chữ số nên dùng mảng 21 phần tử là đủ để lưu kết quả mà không bị tràn giới hạn thanh ghi 16-bit.

- Mảng a dùng để lưu giá trị Fibonacci hiện tại, ban đầu bằng 0, tương ứng với $F(0) = 0$. Mảng b dùng để lưu giá trị Fibonacci kế tiếp, ban đầu bằng 1, tương ứng với $F(1) = 1$. Các số Fibonacci được lưu theo từng chữ số thập phân trong mảng, bắt đầu từ hàng đơn vị; ví dụ số 123 sẽ được lưu theo thứ tự 3, 2, 1.

- Sau khi khai báo dữ liệu, chương trình in thông báo nhập N và đọc lần lượt hai ký tự từ bàn phím bằng INT 21h với chức năng AH = 01h. Vì ký tự nhập từ bàn phím là mã ASCII nên chương trình dùng sub al, '0' để đổi từng ký tự số thành giá trị số. Chữ số thứ nhất được lưu tạm vào BL, sau đó đưa vào AH; chữ số thứ hai nằm trong AL. Lệnh AAD được dùng để đổi hai chữ số này thành một số thập phân theo công thức $N = 10 \times \text{chữ số thứ nhất} + \text{chữ số thứ hai}$, sau đó lưu kết quả vào biến n. Ví dụ nhập 05 thì $N = 5$, nhập 25 thì $N = 25$.

2. Vòng lặp chính và thủ tục in số Fibonacci

```
main_loop:
```

```
call print_a
```

```
call next_fib
```

```
dec n
```

```
jnz main_loop
```

```
mov ax, 4C00h
```

```
int 21h
```

```
print_a proc
```

```
mov bx, DIG-1
```

```
find_digit:
```

```
cmp byte ptr [a+bx], 0
```

```
jne print_loop
```

```
dec bx
```

```

    jns find_digit

    mov dl, '0'
    mov ah, 02h
    int 21h
    call crlf
    ret

print_loop:
    mov ah, 02h

print_digit:
    mov dl, [a+bx]
    add dl, '0'
    int 21h

    dec bx
    jns print_digit

    call crlf
    ret
print_a endp

```

- Vòng lặp chính là phần điều khiển số lần in dãy Fibonacci. Sau khi chương trình đã nhập được giá trị N, biến n sẽ đóng vai trò là bộ đếm, cho biết còn phải in bao nhiêu số Fibonacci nữa. Trong mỗi vòng lặp, chương trình gọi print_a để in số Fibonacci hiện tại đang lưu trong mảng a, sau đó gọi next_fib để tính số Fibonacci kế tiếp. Sau khi in xong một số, lệnh dec n sẽ giảm n đi 1. Nếu sau khi giảm mà n vẫn chưa bằng 0, lệnh jnz main_loop sẽ đưa chương trình quay lại đầu vòng lặp để tiếp tục in số tiếp theo. Ngược lại, nếu n = 0, nghĩa là chương trình đã in đủ N số Fibonacci theo yêu cầu, chương trình sẽ thoát bằng ngắt INT 21h với chức năng AH = 4Ch.

- Thủ tục print_a có nhiệm vụ in giá trị Fibonacci hiện tại đang được lưu trong mảng a. Vì chương trình lưu số theo từng chữ số thập phân và lưu từ hàng đơn vị trước, nên khi in ra màn hình không thể in từ đầu mảng đến cuối mảng. Thay vào đó, chương trình phải bắt đầu kiểm tra từ vị trí cao nhất của mảng, tức DIG - 1, để tìm chữ số khác 0 đầu tiên. Việc này giúp bỏ qua các số 0 dư ở phía trước. Sau khi tìm được chữ số có nghĩa đầu tiên, chương trình in các chữ số theo chiều ngược lại, từ vị trí cao xuống vị trí thấp, để hiển thị đúng thứ tự từ trái sang phải. Nếu toàn bộ mảng a đều bằng 0, nghĩa là số hiện tại là 0, chương trình

sẽ in riêng ký tự 0 ra màn hình. Sau mỗi lần in xong một số Fibonacci, chương trình gọi `crlf` để xuống dòng, giúp mỗi số Fibonacci được hiển thị trên một dòng riêng.

3. Tính Fibonacci kế tiếp và xuống dòng

```
next_fib proc
    xor si, si
    xor bl, bl
    mov cx, DIG

next_loop:
    mov al, [b+si]
    mov dl, al

    add al, [a+si]
    add al, bl

    xor bl, bl

    cmp al, 10
    jb no_carry

    sub al, 10
    inc bl

no_carry:
    mov [a+si], dl
    mov [b+si], al

    inc si
    loop next_loop

    ret
next_fib endp

crlf proc
    mov ah, 02h

    mov dl, 13
    int 21h
```



```
mov dl, 10
int 21h

ret
crlf endp
```

- Thủ tục next_fib tính số Fibonacci kế tiếp sau mỗi lần chương trình in xong giá trị hiện tại. Trong chương trình, a đang lưu số Fibonacci hiện tại, còn b đang lưu số Fibonacci kế tiếp. Vì vậy, để đi sang bước tiếp theo của dãy Fibonacci, chương trình cập nhật theo công thức $A, B = B, A + B$. Điều này có nghĩa là giá trị cũ của B sẽ trở thành giá trị mới của A, còn giá trị mới của B sẽ bằng tổng của A cũ và B cũ.
- Do vi xử lý 8086 chỉ xử lý trực tiếp tốt các giá trị 8-bit hoặc 16-bit, trong khi các số Fibonacci về sau có thể rất lớn, chương trình không lưu toàn bộ số Fibonacci trong một thanh ghi. Thay vào đó, mỗi số Fibonacci được lưu bằng một mảng, trong đó mỗi phần tử của mảng lưu một chữ số thập phân. Vì vậy, khi tính $A + B$, chương trình sẽ cộng lần lượt từng chữ số ở cùng vị trí của hai mảng. Nếu tổng của hai chữ số nhỏ hơn 10 thì lưu trực tiếp kết quả vào mảng b. Nếu tổng lớn hơn hoặc bằng 10 thì chương trình trừ đi 10 và tạo biến nhớ để cộng sang chữ số tiếp theo.
- Trong quá trình tính toán, chương trình dùng SI làm chỉ số để duyệt qua từng phần tử của mảng. Thanh ghi BL được dùng để lưu phần nhớ khi cộng. Thanh ghi AL chứa kết quả cộng tại vị trí hiện tại, còn DL tạm thời giữ giá trị cũ của B để sau đó gán lại cho A. Nhờ vậy, sau mỗi vòng trong thủ tục next_fib, chương trình vừa tính được chữ số mới của B, vừa cập nhật A thành giá trị cũ của B.
- Thủ tục crlf được dùng để xuống dòng sau mỗi lần in một số Fibonacci. Trong môi trường DOS/emu8086, để xuống dòng đúng cách cần in hai ký tự ASCII là 13 và 10. Ký tự 13 đưa con trỏ về đầu dòng, còn ký tự 10 đưa con trỏ xuống dòng mới. Vì vậy, sau khi in xong một số Fibonacci, chương trình gọi crlf để số tiếp theo được hiển thị ở dòng kế tiếp.

III. References

- TS. Vũ Đức Lung, TS. Lê Quang Minh, ThS. Phan Đình Duy, *Giáo trình Vi điều khiển*, Trường Đại học Công nghệ Thông tin - ĐHQG TP.HCM, 2015
- Documentation for emu8086 - assembler and microprocessor emulator.
- Link video 1 (Content 1):
https://drive.google.com/file/d/1_OKx_S94PyZVhWiTQWY6R-3neaggAYD5/view?usp=sharing
- Link video 2 (Exercise 1):
https://drive.google.com/file/d/1EfcRyOnEpgLZFKYa1SF0L7rAkN2_PpdJ/view?usp=sharing